

以 Java 實作 Gemini 函式呼叫，打造確定性生成式 AI

來源：<https://codelabs.developers.google.com/gemini-function-calling-java?hl=zh-tw>

步驟數：9

示範 Java 應用程式中的 Gemini 函式呼叫功能，方法包括透過叫用 Gemini 模型來自動化調度管理函式呼叫的輸入內容、叫用 API，然後在其他 Gemini 呼叫中處理回應，並部署至 REST 端點。

目錄

1. [1. 簡介](#)
2. [2. Gemini 函式呼叫](#)
3. [3. 需求條件](#)
4. [4. 事前準備](#)
5. [5. 實作 Cloud 函式](#)
6. [6. 使用 HelloWorld.java 類別瞭解函式呼叫](#)
7. [7. 部署及測試](#)
8. [8. 清除所用資源](#)
9. [9. 恭喜](#)

1. 簡介

生成式 AI 模型擅長理解自然語言並做出回應。但如果需要精確且可預測的輸出內容，以執行地址標準化等重要工作，該怎麼辦？傳統生成模型有時會對相同提示提供不同的回覆，可能導致不一致。這時，Gemini 的函式呼叫功能就能派上用場，讓您確定地控制 AI 回覆的元素。

本程式碼研究室會以地址自動完成和標準化使用案例，說明這項概念。為此，我們將建構 Java Cloud 函式，執行下列工作：

1. 取得經緯度座標
2. 呼叫 Google 地圖 Geocoding API，取得對應地址
3. 使用 Gemini 1.0 Pro 函數呼叫功能，以我們需要的特定格式，確定地標準化及摘要這些地址

現在就開始吧！

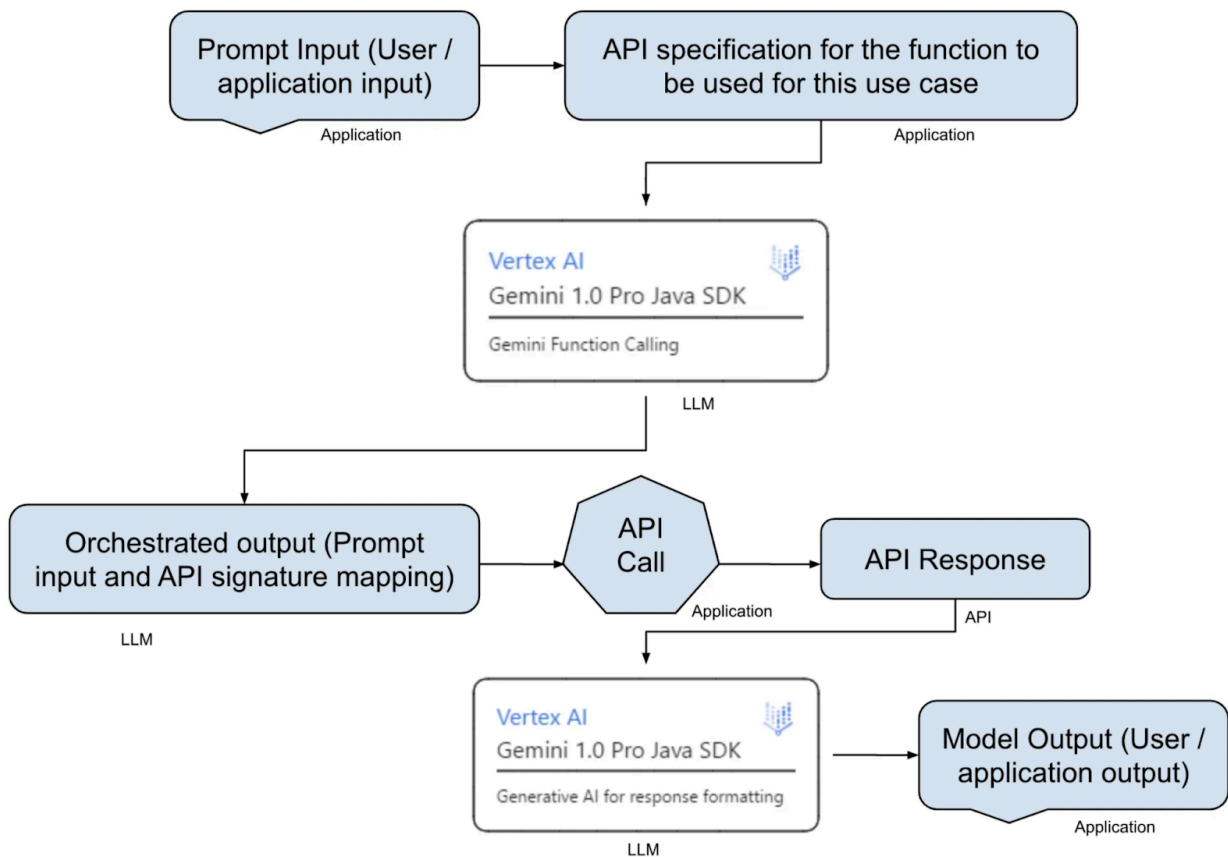
2. Gemini 函式呼叫

在生成式 AI 時代，Gemini 函式呼叫功能之所以獨樹一格，是因為它結合了生成式語言模型的彈性，以及傳統程式設計的精確度。

如要實作 Gemini 函式呼叫，您需要完成下列工作：

- 定義函式：**清楚說明函式。說明必須包含下列資訊：
 - 函式名稱，例如 `getAddress`。
 - 函式預期的參數，例如 `latlng` (字串)。
 - 函式傳回的資料類型，例如地址字串清單。
- 為 Gemini 建立工具：**將函式說明封裝成 API 規格，做為工具。工具就像是專用工具箱，Gemini 可用來瞭解 API 的功能。
- 使用 Gemini 協調 API：**當你傳送提示給 Gemini 時，Gemini 會分析要求，並辨識可使用你提供的工具。接著，Gemini 會執行下列工作，做為智慧協調器：
 - 產生必要的 API 參數，以呼叫您定義的函式。Gemini 不會代您呼叫 API，您必須根據 Gemini 函式呼叫功能為您產生的參數和簽章，呼叫 API。
 - Gemini 會將 API 呼叫的結果回饋到生成程序中，並將結構化資訊納入最終回覆，藉此處理結果。您可以視應用程式需求處理這項資訊。

下圖顯示資料流程、實作步驟，以及每個步驟的擁有者 (例如應用程式、LLM 或 API)：



建構項目

您將建立並部署 Java Cloud 函式，執行下列操作：

- 接受經緯度座標。
 - 呼叫 Google 地圖 Geocoding API，取得對應地址。
 - 使用 Gemini 1.0 Pro 函式呼叫功能，以特定格式標準化及摘要這些地址。
-

步驟 3 · 約 0 分鐘

3. 需求條件

- 瀏覽器，例如 [Chrome](#) 或 [Firefox](#)。
 - 已啟用計費功能的 Google Cloud 專案。
-

4. 事前準備

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud 專案。
2. 確保您的 Google Cloud 雲端專案有啟用計費服務。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 從 Google Cloud 控制台啟用 Cloud Shell。詳情請參閱「[使用 Cloud Shell](#)」。
4. 如果未設定專案，請使用下列指令設定專案：

```
gcloud config set project <YOUR_PROJECT_ID>
```

5. 在 Cloud Shell 中設定下列環境變數：

```
export GCP_PROJECT=<YOUR_PROJECT_ID>  
export GCP_REGION=us-central1
```

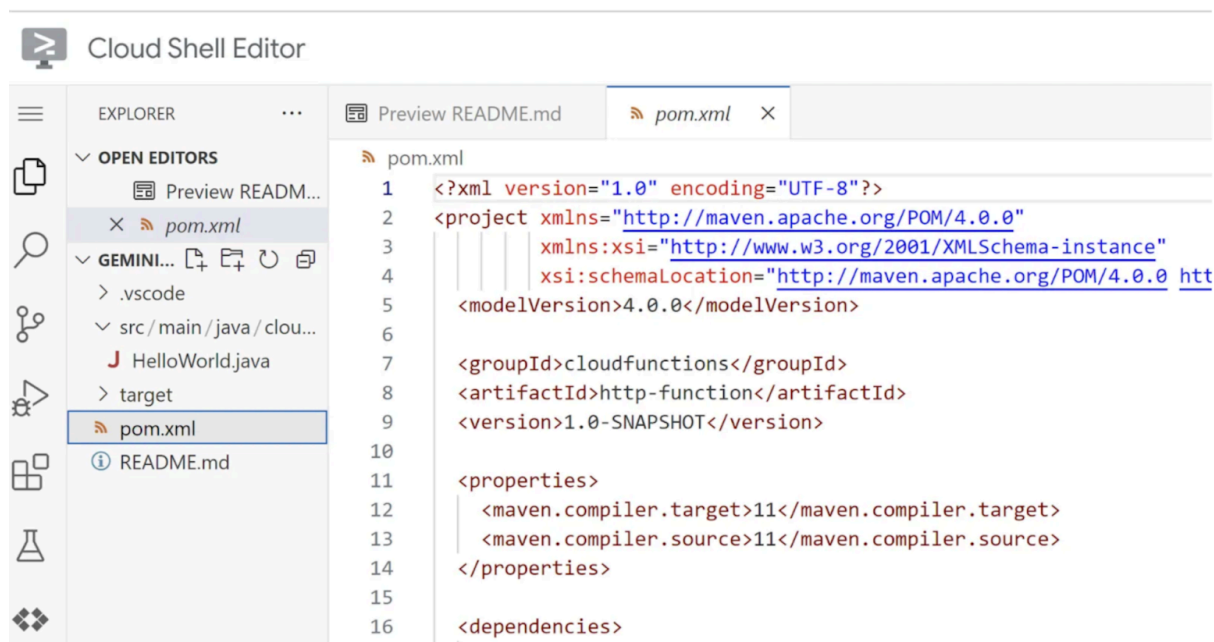
6. 在 Cloud Shell 中執行下列指令，啟用必要的 Google Cloud API：

```
gcloud services enable cloudbuild.googleapis.com cloudfunctions.googleapis.com  
run.googleapis.com logging.googleapis.com storage-component.googleapis.com  
cloudai.companion.googleapis.com aiplatform.googleapis.com
```

7. 開啟 [Cloud Shell 編輯器](#)，按一下「Extensions」(擴充功能)，然後安裝 [Gemini + Google Cloud Code 擴充功能](#)。
-

5. 實作 Cloud 函式

1. 啟動 [Cloud Shell 編輯器](#)。
2. 按一下「Cloud Code」，然後展開「Cloud Functions」部分。
3. 按一下「Create Function」(建立函式) (+) 圖示。
4. 在「Create New Application」(建立新的應用程式) 對話方塊中，選取「Java: Hello World」(Java : Hello World) 選項。
5. 在專案路徑中提供專案名稱，例如 `GeminiFunctionCalling`。
6. 按一下「Explorer」查看專案結構，然後開啟 `pom.xml` 檔案。下圖顯示專案結構：



6. 在 `pom.xml` 檔案的 `<dependencies>... </dependencies>` 標記中新增必要的依附元件。您可以從這個專案的 [GitHub 存放區](#) 存取整個 `pom.xml`。從該處將 `pom.xml` 複製到您正在編輯的目前專案 `pom.xml` 檔案。
7. 從 [GeminiFunctionCalling GitHub](#) 連結複製 `HelloWorld.java` 類別。您必須將 `API_KEY` 和 `project_id` 分別更新為 [地理編碼 API 金鑰](#) 和 Google Cloud 專案 ID。

步驟 6 · 約 0 分鐘

6. 使用 HelloWorld.java 類別瞭解函式呼叫

提示詞輸入

在本例中，輸入提示詞如下：*What's the address for the latlong value 40.714224,-73.961452*。

以下是檔案中與輸入提示詞對應的程式碼片段：

```
String promptText = "What's the address for the latlong value '" + latlngString + "'?";  
//40.714224,-73.961452
```

API 規格

本範例使用 Reverse Geocoding API。API 規格如下：

```
/* Declare the function for the API to invoke (Geo coding API) */  
FunctionDeclaration functionDeclaration =  
    FunctionDeclaration.newBuilder()  
        .setName("getAddress")  
        .setDescription("Get the address for the given latitude and longitude value.")  
        .setParameters(  
            Schema.newBuilder()  
                .setType(Type.OBJECT)  
                .putProperties(  
                    "latlng",  
                    Schema.newBuilder()  
                        .setType(Type.STRING)  
                        .setDescription("This must be a string of latitude and longitude  
coordinates separated by comma")  
                        .build()  
                ).addRequired("latlng")  
                .build()  
        ).build();
```

使用 Gemini 協調提示

提示輸入內容和 API 規格會傳送至 Gemini：

```
// Add the function to a "tool"
Tool tool = Tool.newBuilder()
    .addFunctionDeclarations(functionDeclaration)
    .build();

// Invoke the Gemini model with the use of the tool to generate the API parameters from the
// prompt input.
GenerativeModel model = GenerativeModel.newBuilder()
    .setModelName(modelName)
    .setVertexAi(vertexAI)
    .setTools(Arrays.asList(tool))
    .build();
GenerateContentResponse response = model.generateContent(promptText);
Content responseJSONCnt = response.getCandidates(0).getContent();
```

這項作業的回應是 API 的協調參數 JSON。以下是輸出內容範例：

```
role: "model"
parts {
  function_call {
    name: "getAddress"
    args {
      fields {
        key: "latlng"
        value {
          string_value: "40.714224,-73.961452"
        }
      }
    }
  }
}
```

將下列參數傳遞至 `Reverse Geocoding` API：`"latlng=40.714224,-73.961452"`

將協調結果與 `"latlng=VALUE"` 格式相符。

叫用 API

以下是呼叫 API 的程式碼部分：

```
// Create a request
String url = API_STRING + "?key=" + API_KEY + params;
java.net.http.HttpRequest request = java.net.http.HttpRequest.newBuilder()
    .uri(URI.create(url))
    .GET()
    .build();
// Send the request and get the response
java.net.http.HttpResponse<String> httpresponse = client.send(request,
java.net.http.HttpResponse.BodyHandlers.ofString());
// Save the response
String jsonResult = httpresponse.body().toString();
```

字串 `jsonResult` 包含反向 Geocoding API 的回應。以下是輸出內容的格式化版本：

```
"...277 Bedford Ave, Brooklyn, NY 11211, USA; 279 Bedford Ave, Brooklyn, NY 11211, USA; 277
Bedford Ave, Brooklyn, NY 11211, USA;..."
```

處理 API 回應並準備提示

下列程式碼會處理 API 的回應，並準備提示，內含如何處理回應的說明：

```

// Provide an answer to the model so that it knows what the result
// of a "function call" is.
String promptString =
    "You are an AI address standardizer for assisting with standardizing addresses
accurately. Your job is to give the accurate address in the standard format as a JSON object
containing the fields DOOR_NUMBER, STREET_ADDRESS, AREA, CITY, TOWN, COUNTY, STATE, COUNTRY,
ZIPCODE, LANDMARK by leveraging the address string that follows in the end. Remember the
response cannot be empty or null. ";

Content content =
    ContentMaker.fromMultiModalData(
        PartMaker.fromFunctionResponse(
            "getAddress",
            Collections.singletonMap("address", formattedAddress)));
String contentString = content.toString();
String address = contentString.substring(contentString.indexOf("string_value: \") +
"string_value: \".length(), contentString.indexOf("'", contentString.indexOf("string_value:
\").length() + "string_value: \".length()));

List<SafetySetting> safetySettings = Arrays.asList(
    SafetySetting.newBuilder()
        .setCategory(HarmCategory.HARM_CATEGORY_HATE_SPEECH)
        .setThreshold(SafetySetting.HarmBlockThreshold.BLOCK_ONLY_HIGH)
        .build(),
    SafetySetting.newBuilder()
        .setCategory(HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT)
        .setThreshold(SafetySetting.HarmBlockThreshold.BLOCK_ONLY_HIGH)
        .build()
);

```

叫用 Gemini 並傳回標準化地址

下列程式碼會將上一步驟處理的輸出內容做為提示，傳送給 Gemini：

```

GenerativeModel modelForFinalResponse = GenerativeModel.newBuilder()
    .setModelName(modelName)
    .setVertexAi(vertexAI)
    .build();
GenerateContentResponse finalResponse =
modelForFinalResponse.generateContent(promptString + ": " + address, safetySettings);
System.out.println("promptString + content: " + promptString + ": " + address);
// See what the model replies now
System.out.println("Print response: ");
System.out.println(finalResponse.toString());
String finalAnswer = ResponseHandler.getText(finalResponse);
System.out.println(finalAnswer);

```

`finalAnswer` 變數包含 JSON 格式的標準化地址。以下是輸出範例：

```
{"replies":["{ \"DOOR_NUMBER\": null, \"STREET_ADDRESS\": \"277 Bedford Ave\", \"AREA\":  
\"Brooklyn\", \"CITY\": \"New York\", \"TOWN\": null, \"COUNTY\": null, \"STATE\": \"NY\",  
\"COUNTRY\": \"USA\", \"ZIPCODE\": \"11211\", \"LANDMARK\": null} null}"]}
```

您已瞭解 Gemini 函式呼叫功能如何搭配地址標準化用途運作，現在可以繼續部署 Cloud 函式。

7. 部署及測試

1. 如果您已建立 *GeminiFunctionCalling* 專案並實作 Cloud Function，請繼續執行步驟 2。如果尚未建立專案，請前往 Cloud Shell 終端機，複製這個存放區：`git clone` <https://github.com/AbiramiSukumaran/GeminiFunctionCalling>
2. 前往專案資料夾：`cd GeminiFunctionCalling`
3. 執行下列陳述式，建構及部署 Cloud 函式：

```
gcloud functions deploy gemini-fn-calling --gen2 --region=us-central1 --runtime=java11 --source=. --entry-point=cloudcode.helloworld.HelloWorld --trigger-http
```

部署完成後的網址格式如下：https://us-central1-YOUR_PROJECT_ID.cloudfunctions.net/gemini-fn-calling

4. 在終端機中執行下列指令，測試 Cloud 函式：

```
gcloud functions call gemini-fn-calling --region=us-central1 --gen2 --data '{"calls": [{"40.714224,-73.961452}"]}'
```

以下是隨機範例提示的回應：`'{"replies":[{"DOOR_NUMBER": "277", "STREET_ADDRESS": "Bedford Ave", "AREA": null, "CITY": "Brooklyn", "TOWN": null, "COUNTY": "Kings County", "STATE": "NY", "COUNTRY": "USA", "ZIPCODE": "11211", "LANDMARK": null}]}``'`

8. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後按一下「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關閉) 刪除專案。
 4. 如要保留專案，請略過上述步驟，然後前往 Cloud Functions，從函式清單中勾選要刪除的函式，並按一下「刪除」。
-

步驟 9 · 約 0 分鐘

9. 恭喜

恭喜！您已在 Java 應用程式中成功使用 Gemini 函式呼叫功能，並將生成式 AI 工作轉換為確定性且可靠的程序。如要進一步瞭解可用的模型，請參閱 [Vertex AI LLM 產品說明文件](#)。
